

Security Before the Finish Line

Bradley D. Saucier

Southern New Hampshire University

CS 405: Secure Coding

Professor Aaron Demory

August 20, 2026

One of the most useful parts of CS 405 was taking the phrase "secure coding" apart. It is easy to agree with and much harder to turn into daily decisions. Project One made me define ten C++ standards for Green Pace, connect each one to a risk, and identify a way to check it. Project Two made me explain what evidence would show that the policy was actually being followed. After doing both, I no longer see security as a final inspection. It has to shape the code, the tests, and the process used to approve changes.

A secure coding standard gives that work a common baseline. The Green Pace policy uses the SEI CERT C++ Coding Standard for rules involving integer conversion, unsigned arithmetic, strings, SQL injection, memory lifetime, assertions, exceptions, initialization, deallocation, and container bounds. This is more useful than telling developers to "write safe code." For example, STD-010-CPP requires validation before container access and recommends checked access with `at()` when practical. That rule leads directly to a testable behavior.

The vector tests from Project Two made the difference concrete. One test requests the index equal to size, and another requests index zero from an empty vector. Both expect `std::out_of_range` instead of hoping the access succeeds. Those are small tests, but they examine the exact boundaries where assumptions fail. The portfolio application follows the same approach. It limits file size and word count, accepts only its documented ASCII format, rejects empty evidence, uses standard containers, and reports controlled errors. These choices were made before packaging the program, not added as a cleanup step.

That approach matches the NIST Secure Software Development Framework, which places secure practices throughout the software life cycle instead of in one review before release (Souppaya et al., 2022). Early controls do add work. Validation paths and negative tests take

longer than writing only the success case. Still, that cost is predictable while the project is small. A defect found after release may require redesign, emergency testing, incident response, and an explanation to users. I would rather account for security work in the plan than discover its cost during an incident.

Risk assessment helps decide which controls deserve attention first. In the Green Pace policy, SQL injection and memory protection are Level 5 risks. Unsigned arithmetic and container bounds are Level 4. The rankings are based on severity, likelihood, exposure, detectability, and remediation cost. NIST SP 800-30 describes risk assessment as support for choosing responses and allocating resources, which is how I used the matrix rather than treating it as a scorecard (Joint Task Force Transformation Initiative, 2012).

The cost-benefit case for testing is straightforward. Writing fifteen vector tests took more effort than accepting a successful compilation, but the tests can run after every later change. The portfolio repository now adds tests for tokenization, repeated-word shortages, missing files, input limits, and both possible and impossible source claims. Local verification passed all 31 tests with compiler warnings treated as errors. That result does not prove the program has no defects. It does show that the documented behaviors have repeatable checks. Risk ratings also remain judgment calls, not exact financial measurements. Some risk will be accepted, so the Green Pace exception process requires an owner, rationale, compensating control, approval, and expiration date. That is a better decision than allowing an unresolved risk to disappear into a backlog.

Zero trust changed how I think about those assumptions. NIST SP 800-207 removes implicit trust based on location and focuses decisions on users, assets, and resources (Rose et al., 2020). Green Pace applies that idea through default deny, least privilege, authentication, authorization,

and accounting. The text checker is not a zero trust architecture, but the same habit is useful at the code level. A file is not safe merely because it is local. The program checks that it is a regular file, enforces limits while reading it, rejects unsupported input, and stops when validation fails. The GitHub workflows also begin with read-only repository permissions. Trust follows a check and a stated need.

The final lesson is that a policy needs enforcement and feedback. A document can define the right rules and still be ignored. The repository is configured to build with GCC, Clang, and MSVC, run the tests, use sanitizers, validate the portfolio files, and perform CodeQL analysis. The action references and GoogleTest download are pinned so the build does not silently accept different third-party code. These checks make the policy harder to overlook, but they still need judgment. Tests cover only cases someone wrote. Static analysis may report false positives. A green workflow is evidence, not proof.

My recommendation is to keep each required check tied to a specific rule, assign someone to review failures, and document exceptions instead of bypassing the pipeline. That makes security visible without turning every warning into the same emergency. The Green Pace work showed me that secure coding is not mainly about adding more tools. It is about replacing assumptions with limits, tests, and decisions that another person can review.

References

- Joint Task Force Transformation Initiative. (2012). Guide for conducting risk assessments (NIST Special Publication 800-30 Rev. 1). National Institute of Standards and Technology.
<https://doi.org/10.6028/NIST.SP.800-30r1>
- Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero trust architecture (NIST Special Publication 800-207). National Institute of Standards and Technology.
<https://doi.org/10.6028/NIST.SP.800-207>
- Software Engineering Institute. (n.d.). SEI CERT C++ coding standard. Carnegie Mellon University.
<https://wiki.sei.cmu.edu/confluence/pages/viewpage.action?pageId=88046682>
- Souppaya, M., Scarfone, K., & Dodson, D. (2022). Secure Software Development Framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities (NIST Special Publication 800-218). National Institute of Standards and Technology.
<https://doi.org/10.6028/NIST.SP.800-218>